

Sprint Task Breakdown

Task 1: Schema & Server Setup (Backend Only)

1.1 Add reserved_by Column to listings Table

What it does: Adds column to track which user reserved an item.

Why needed: Simplifies "Create Transaction" page queries — fetch reserved items with ?reserved_by=me.

File: Run in Supabase SQL Editor

Code:

```
ALTER TABLE listings ADD COLUMN reserved_by UUID REFERENCES profiles(id);
```

Done when: Column exists in Supabase.

1.2 Register New Routes in server.js

What it does: Tell Express about new route files.

File: server.js:~93

Add after line 94:

```
const transactionsRouter = require("./routes/transactions");
const ratingsRouter = require("./routes/ratings");
app.use("/transactions", transactionsRouter);
app.use("/ratings", ratingsRouter);
```

Done when: Server starts without errors.

Task 2: Purchase/Reserve Flow (Full Stack)

2.1 (Backend) Update Reserve Endpoint — Set reserved_by

What it does: When buyer reserves item, tag listing with their user ID.

File: routes/listings.js:473-478

Change:

```
.update({
  status: "reserved",
  reserved_by: buyerId // ← ADD
})
```

Done when: After reserving, listings.reserved_by = buyerId in Supabase.

2.2 (Backend) Update Search Endpoint — Filters + Seller Join

What it does: Accept status/reserved_by params + join seller info for ratings.

File: routes/listings.js:232-273

Changes:

```
// Line ~238: Change select
.select(`
  *,
  profiles:user_id (name),
  ratings:ratings (rating)
`);

// After line ~257, add:
if (status && status !== "All") query = query.eq("status", status);
if (reserved_by && reserved_by !== "All") query = query.eq("reserved_by",
reserved_by);
```

Done when: GET /listings/search?status=available returns items with seller name + ratings.

2.3 (Frontend) Update components/card.js

What it does: Refresh after reserve + show real seller name/rating.

File: components/card.js

Change 1 — Lines 148-153 (after reserve success):

```
setTimeout(() => { window.location.reload(); }, 500);
```

Change 2 — Lines 51-54 (seller info):

```
Sold by: ${listing.profiles?.name || "Unknown"}<br>
★ ${listing.ratings?.length > 0
  ? (listing.ratings.reduce((s, r) => s + r.rating, 0) /
    listing.ratings.length).toFixed(1)
  : "No ratings yet"}
```

Done when: Cards show seller name + rating. After reserving, dashboard refreshes.

Task 3: My Listings Page (Frontend)

3.1 Create my-listings.html + my-listings.js

What it does: New page showing user's listings with edit/delete.

Why needed: Dashboard "My Listings" link points to #. User story requires delete functionality.

Files: pages/my-listings.html (NEW), scripts/my-listings.js (NEW)

Checklist:

- ☐ Sidebar with student nav links
- ☐ Fetch: GET /listings/search?user_id=me (needs Task 2.2)
- ☐ Each card: image, title, status, Edit button, Delete button
- ☐ Delete: confirm dialog → DELETE /listings/:id
- ☐ Prevent deleting if active transaction exists

Hint: Reuse createListingCard() from card.js, add delete button in actions div.

Done when: Page shows user's listings. Edit and Delete work.

Task 4: Transactions System (Full Stack)

4.1 (Backend) Create routes/transactions.js

What it does: New file with all transaction endpoints.

Why needed: transactions table has no routes. Current frontend uses mock data.

File: routes/transactions.js (NEW)

Endpoints:

Method	Endpoint	Purpose
GET	/transactions?user_id=&status=	Fetch user's transactions (join listings, profiles, slots). Query: buyer_id = user_id OR seller_id = user_id
POST	/transactions	Book slot for reserved item. Body: { listing_id, slot_id }. Set status = "in_progress"
PUT	/transactions/:id/cancel	Cancel transaction. Both buyer and seller can cancel. Set status = "cancelled", revert listing to available, clear reserved_by

Key logic for GET:

```
let query = supabase
  .from("transactions")
  .select(`
    *,
    listings (*),
    buyer:profiles!buyer_id (name),
    seller:profiles!seller_id (name)
  `);
if (user_id) {
  query = query.or(`buyer_id.eq.${user_id},seller_id.eq.${user_id}`);
}
```

Key logic for Cancel (both buyer AND seller can cancel):

```
const { data: tx } = await supabase
  .from("transactions")
  .select("*")
  .eq("id", id)
  .single();
// Both buyer and seller can cancel
if (tx.buyer_id !== currentUserId && tx.seller_id !== currentUserId) {
  return res.status(403).json({ error: "Not authorized to cancel this transaction" });
}
// Proceed with cancellation...
```

Key logic for POST (slot booking → status = "in_progress"):

```
const { data: updated } = await supabase
  .from("transactions")
  .update({
    slot_id,
    status: "in_progress" // ← Both sale and trade
  })
  .eq("id", transaction.id)
  .select("*");
```

Done when: All endpoints return JSON and modify Supabase correctly.

4.2 (Frontend) Create Student Transactions Page (NEW — Separate from Staff)

What it does: New student-facing page to view/manage transactions. Staff keep their existing transactions.html unchanged.

Files: pages/student-transactions.html (NEW), scripts/student-transactions.js (NEW)

Checklist:

- ☐ Sidebar: student nav (Dashboard, My Listings, Messages, Ratings)
- ☐ Status tabs: All / Pending / In Progress / Completed / Cancelled
- ☐ Fetch: GET /transactions?user_id=\${currentUserId} (uses Task 4.1)

- ☐ Cards show: item image, title, buyer/seller name, status badge
- ☐ Non-completed → "Cancel" button → PUT /transactions/:id/cancel
- ☐ Completed → "Rate" button → /rating.html?transactionId=&sellerId=
- ☐ "New Transaction" button → /create-transaction.html

Done when: Students see real transactions, can cancel (both parties), can rate.

4.3 (Frontend) Create Transaction Page

What it does: Page where buyer selects reserved item + slot to finalize transaction.

Files: pages/create-transaction.html (NEW), scripts/create-transaction.js (NEW)

Checklist:

- ☐ Dropdown: "Select Reserved Item" — fetch from GET /transactions?user_id=me&status=pending
- ☐ Dropdown: "Select Trade Slot" — fetch from GET /slots
- ☐ Submit: "Book Slot & Create Transaction" → POST /transactions (uses Task 4.1)
- ☐ On success: redirect to /student-transactions.html

Done when: User can select reserved item → pick slot → create transaction.

Task 5: Ratings System (Full Stack)

5.1 (Backend) Create routes/ratings.js

What it does: Handle rating submission and fetching.

File: routes/ratings.js (NEW)

Endpoints:

Method	Endpoint	Purpose
POST	/ratings	Submit rating. Body: { rated_id, transaction_id, rating, review }. Get rater_id from session!
GET	/ratings?user_id=	Return ratings for a user + average rating

Security note: Never trust rater_id from request body — get from session.

Done when: POST /ratings inserts into table. GET /ratings?user_id=x returns data.

5.2 (Frontend) Update Rating Page

What it does: Connect rating.html to real backend.

Files: pages/rating.html, scripts/rating.js

Change in rating.js:47-70:

```
submitBtn.addEventListener("click", async () => {
  const res = await fetch("/ratings", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({
      rated_id: sellerId,
      transaction_id: transactionId,
      rating: selectedRating,
      review: reviewText.value.trim()
    })
  });

  const data = await res.json();
  // Handle success/error...
});
```

Done when: Submitting rating inserts into Supabase ratings table.

Task 6: Dashboard Fixes (Frontend)

6.1 Update pages/dashboard.html

What it does: Fix broken links, hide reserved items, fix hardcoded stats.

File: pages/dashboard.html

Checklist:

- ☐ Line 257: Fix "My Listings" link to /my-listings.html
- ☐ Line 259: Fix "Ratings" link to /rating.html (or rename file to rate-user.html)
- ☐ Inline script ~line 410: Add status=available to hide reserved items:

```
const response = await fetch("/listings/search?status=available");
```

- ☐ Lines 306-314: Remove hardcoded "Items Sold: 12" and "Pending Trades: 02"

Done when: Dashboard only shows available items. Links work. Stats are removed.

Task 7: History Page Fix (Frontend)

7.1 Fix scripts/history.js

What it does: Remove hardcoded localhost:3000 URL.

File: scripts/history.js

Changes:

```
// Remove line 1: const API = "http://localhost:3000";  
// Line 10: Change to:  
const res = await fetch(`/history/${userId}`);
```

Done when: History page works without hardcoded URL.

Execution Order

Phase 1: Backend Foundation

1. Task 1 — Schema + Route Registration
2. Task 2.1 + 2.2 — Reserve + Search Endpoints
3. Task 4.1 — Transactions Routes
4. Task 5.1 — Ratings Routes

Phase 2: Frontend (after backend ready)

1. Task 2.3 — Update card.js
2. Task 3 — My Listings Page
3. Task 4.2 + 4.3 — Student Transactions Pages
4. Task 5.2 — Rating Page
5. Task 6 — Dashboard Fixes
6. Task 7 — History Fix

Quick Reference: Task → File Map

Task	File(s)	Type
1.1	Supabase SQL	Schema
1.2	server.js:~93	Modify
2.1	routes/listings.js:473-478	Modify
2.2	routes/listings.js:232-273	Modify

2.3	components/card.js	Modify
3.1	pages/my-listings.html, scripts/my-listings.js	NEW
4.1	routes/transactions.js	NEW
4.2	pages/student-transactions.html, scripts/student-transactions.js	NEW
4.3	pages/create-transaction.html, scripts/create-transaction.js	NEW
5.1	routes/ratings.js	NEW
5.2	pages/rating.html, scripts/rating.js	Modify
6.1	pages/dashboard.html	Modify
7.1	scripts/history.js	Modify